
Semantic-Weighted Pose Estimation in ORB-SLAM3 Relocalization

Technical Lecture Notes

ORB_SLAM3_Relocalization Project · April 2026

Abstract. These notes describe the semantic-weighted Pose-from- n -Points (PnP) algorithm implemented in the `orb_slam3_relocalization` package. The core idea is to assign *higher trust* to ORB keypoints that fall inside YOLO-detected bounding boxes of permanent scene structures (doors), and *lower trust* to background keypoints. The resulting weighted Levenberg–Marquardt optimization on $SE(3)$ produces more stable and accurate pose estimates than standard unweighted RANSAC-PnP.

Contents

1	System Overview	2
2	Camera Model	2
3	ORB Feature Extraction and Candidate Matching	3
4	YOLO Detection Integration	4
4.1	Detection Message Format	4
4.2	ROS 2 Subscription and Parsing	4
4.3	Bounding Box Coordinate Scaling	5
5	Landmark Segmentation	5
6	Semantic Weight Assignment	6
6.1	Weight Hierarchy	6
6.2	Semantic Weight Function	6
6.3	Weight Assignment to All Matches	6
7	Weighted PnP — Mathematical Formulation	7
7.1	Objective Function	7
7.2	SE(3) Lie Group Representation	8
7.3	Weight Normalisation	8
8	Projection Jacobian	8
9	Levenberg–Marquardt Solver	9
9.1	Weighted Normal Equations	9
9.2	Core Optimization Loop	10
9.3	Solver Parameters	11
10	Result Structure and Metrics	11
11	Full Algorithm Flowchart	13

§1 System Overview

The relocalization system localizes a live camera against a pre-built ORB-SLAM3 map. Four ROS 2 nodes cooperate to achieve this:

- **camera_node** — streams raw frames on `/camera/image_raw` (`sensor_msgs/Image`).
- **yolo_bbox_node** — runs YOLO inference and publishes JSON detections on `yolo/results` at a fixed resolution of 640×480 px.
- **relocalization_node** — subscribes to both topics, extracts ORB features, applies semantic weights from YOLO bboxes, runs a weighted PnP solver, and publishes the pose on `/relocalization/pose`.
- **navigation_node** — plans paths using the published pose.

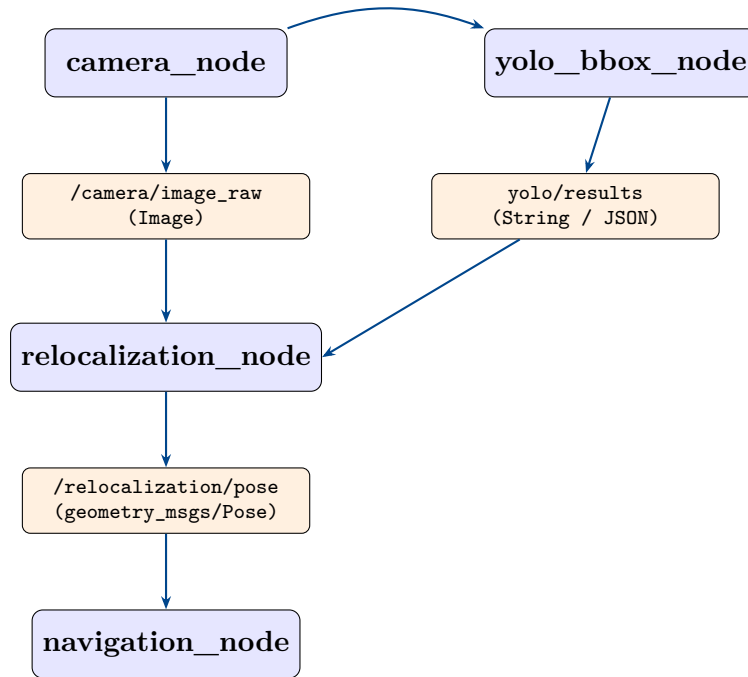


Figure 1: ROS2 node graph. The **relocalization_node** fuses raw image data with YOLO semantic detections to produce a weighted pose estimate.

§2 Camera Model

The system uses a standard **pinhole camera model**. A 3-D world point $\mathbf{X}^w$ is first transformed into camera coordinates $\mathbf{P}_c = [X_c, Y_c, Z_c]^\top$ by the current pose $\mathbf{T}_{cw}$, then projected onto the image plane:

Perspective Projection

$$u = f_x \frac{X_c}{Z_c} + c_x, \quad v = f_y \frac{Y_c}{Z_c} + c_y \quad (1)$$

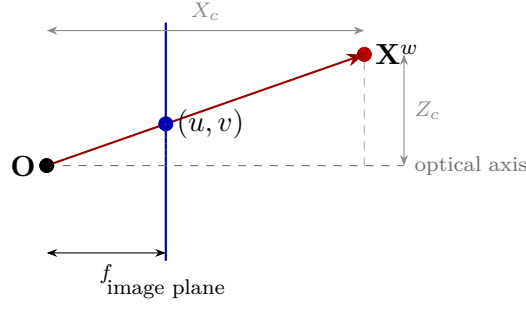


Figure 2: Pinhole projection geometry. The world point $\mathbf{X}^w$ projects to pixel (u, v) at the intersection of the ray from optical centre $\mathbf{O}$ and the image plane.

Table 1: Camera intrinsics used — TUM FR3 dataset (`tum_fr3.yaml`)

YAML key	Symbol	Value	Description
<code>Camera1.fx</code>	f_x	535.4	Horizontal focal length (px)
<code>Camera1.fy</code>	f_y	539.2	Vertical focal length (px)
<code>Camera1.cx</code>	c_x	320.1	Horizontal principal point (px)
<code>Camera1.cy</code>	c_y	247.6	Vertical principal point (px)
<code>Camera.width</code>	W	640	Image width (px)
<code>Camera.height</code>	H	480	Image height (px)
<code>Camera1.k1-k3</code>	—	0.0	Radial distortion (none)

§3 ORB Feature Extraction and Candidate Matching

Each frame is processed in three steps before any pose computation:

1. **ORB extraction.** 1000 oriented FAST corners are detected across an 8-level image pyramid (scale factor 1.2) and described with 256-bit binary ORB descriptors.
2. **Bag-of-Words retrieval.** The frame’s BoW vector is compared against all map keyframes. Keyframes whose cosine similarity exceeds `BowSimilarityThreshold` = 0.018 become *candidates*.
3. **Descriptor matching.** ORB descriptors of the query frame are matched to each candidate keyframe, yielding a set of 2D–3D correspondences $\{(\mathbf{p}_i, \mathbf{X}_i^w)\}_{i=1}^n$.

Table 2: ORB extractor and relocalization parameters (`tum_fr3.yaml`)

Parameter	Value	Description
<code>ORBextractor.nFeatures</code>	1000	Keypoints extracted per frame
<code>ORBextractor.scaleFactor</code>	1.2	Pyramid level scale ratio
<code>ORBextractor.nLevels</code>	8	Number of pyramid levels
<code>ORBextractor.iniThFAST</code>	20	Initial FAST response threshold
<code>ORBextractor.minThFAST</code>	7	Fallback FAST threshold
<code>Relocalization.BowSimilarityThreshold</code>	0.018	Minimum BoW cosine similarity
<code>Relocalization.MinInliers</code>	15	Minimum inliers for pose acceptance

§4 YOLO Detection Integration

4.1 Detection Message Format

The `yolo_bbox_node` runs YOLOv8 inference on each incoming frame (resized to 640×480) and publishes results as a JSON string on `yolo/results`:

```

1 {
2   "frame_id": 42,
3   "detections": [
4     {
5       "cls_id": 164,
6       "conf_score": 0.92,
7       "bbox_coords": { "x1": 100, "y1": 200, "x2": 350, "y2": 450 }
8     }
9   ]
10 }
```

Listing 1: JSON payload on `yolo/results` (`yolo_bbox_video.py`)

Note

Class ID **164** = **door** in the YOLOv8 Open Images V7 vocabulary. Doors are chosen because they are *permanent*, *geometrically stable*, and *visually distinctive* — ideal anchor points for relocalization.

4.2 ROS 2 Subscription and Parsing

The relocalization node subscribes to `yolo/results` with a mutex-protected callback. Parsed bounding boxes are stored in `latest_bboxes_`:

```

1 void yoloCallback(const std_msgs::msg::String::SharedPtr msg)
2 {
3     try {
4         auto data = nlohmann::json::parse(msg->data);
5         std::lock_guard<std::mutex> lock(bbox_mutex_);
6         latest_bboxes_.clear();
7
8         for (const auto &det : data["detections"])
9         {
10             int x1 = det["bbox_coords"]["x1"].get<int>();
11             int y1 = det["bbox_coords"]["y1"].get<int>();
12             int x2 = det["bbox_coords"]["x2"].get<int>();
13             int y2 = det["bbox_coords"]["y2"].get<int>();
14
15             latest_bboxes_.push_back({
16                 det["cls_id"].get<int>(),
17                 cv::Rect(x1, y1, x2 - x1, y2 - y1)
18             });
19         }
20     }
21     catch (const nlohmann::json::exception &e) {
22         RCLCPP_WARN(get_logger(), "YOLO parse error: %s", e.what());
23     }
24 }
```

Listing 2: `yoloCallback()` — `relocalization_node.cpp`, lines 450–467

Note

`bbox_mutex_` ensures thread safety between the YOLO ROS 2 callback (which writes `latest_bboxes_`) and the main processing loop (which reads it).

4.3 Bounding Box Coordinate Scaling

YOLO always outputs coordinates at 640×480 . Before use, these are scaled to the processing resolution ($W_{\text{proc}} \times H_{\text{proc}}$, also 640×480 in the TUM FR3 configuration, but handled generically):

Bbox Scaling Factors

$$s_x = \frac{W_{\text{proc}}}{640}, \quad s_y = \frac{H_{\text{proc}}}{480} \quad (2)$$

Each bbox corner (x, y) maps to $(s_x x, s_y y)$.

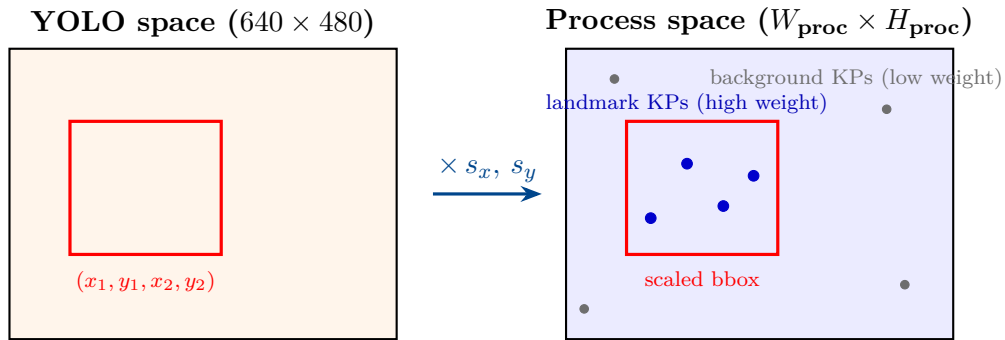


Figure 3: YOLO bboxes are scaled from 640×480 to the processing resolution. ORB keypoints inside the scaled box become *landmark keypoints* with high weight; all others are *background*.

5 Landmark Segmentation

After scaling, the node iterates over all ORB matches and assigns each matched 2-D point to a `LandmarkRegion` when it falls inside a scaled bounding box. A pixel proximity tolerance of 0.5px is used for floating-point safety:

```

1  const float sx = (float)processSize.width / 640.0f;
2  const float sy = (float)processSize.height / 480.0f;
3
4  for (const auto &det : bboxes)
5  {
6      Relocalization::LandmarkRegion region;
7      region.cls_id = det.cls_id;
8      region.bbox = cv::Rect(
9          (int)(det.bbox.x * sx), (int)(det.bbox.y * sy),
10         (int)(det.bbox.width * sx), (int)(det.bbox.height * sy));
11
12     for (const auto &kp : result.queryKeypoints)
13     {
14         if (region.bbox.contains(cv::Point((int)kp.pt.x, (int)kp.pt.y)))

```

```

15         region.keypoints.push_back(kp);
16     }
17
18     result.landmarkRegions.push_back(std::move(region));
19 }

```

Listing 3: Landmark segmentation — `relocalization_node.cpp`, lines 171–203

The `LandmarkRegion` struct groups all keypoints belonging to one detection:

```

1 struct LandmarkRegion
2 {
3     int                cls_id;        // YOLO class ID (e.g. 164 =
4         door)
5     cv::Rect           bbox;          // Bounding box (process
6         resolution)
7     std::vector<cv::KeyPoint> keypoints; // ORB keypoints inside the
8         bbox
9 };

```

Listing 4: `LandmarkRegion` struct — `relocalization.h`, lines 21–27

§6 Semantic Weight Assignment

6.1 Weight Hierarchy

Three tiers of trust are defined based on YOLO class:

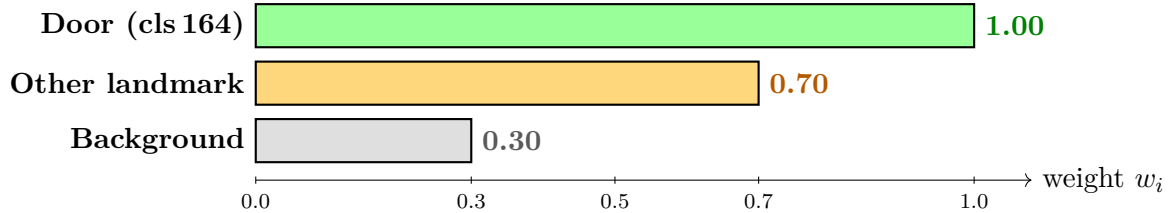


Figure 4: Semantic weight tiers. Door keypoints receive maximum weight (1.00); other detected landmarks get 0.70; background keypoints get 0.30.

6.2 Semantic Weight Function

```

1 static float getSemanticWeight(int cls_id)
2 {
3     switch (cls_id)
4     {
5         case 164: return 1.0f; // Door -- highly permanent landmark
6         default: return 0.7f; // Any other detected object class
7     }
8 }

```

Listing 5: `getSemanticWeight()` — `relocalization.cpp`, lines 1113–1121

6.3 Weight Assignment to All Matches

```

1 std::vector<float> RelocalizationModule::assignWeightsFromLandmarks(
2     const std::vector<cv::Point2f> &matched2DPoints,
3     const std::vector<LandmarkRegion>&landmarkRegions,
4     float permanentWeight,    // = 1.0f at call site
5     float backgroundWeight)   // = 0.3f at call site
6 {
7     // Initialise every point to background level
8     std::vector<float> weights(matched2DPoints.size(), backgroundWeight)
9     ;
10
11     for (size_t i = 0; i < matched2DPoints.size(); ++i)
12     {
13         for (const auto &region : landmarkRegions)
14         {
15             for (const auto &kp : region.keypoints)
16             {
17                 // Pixel proximity: 0.5 px tolerance
18                 if (std::abs(kp.pt.x - matched2DPoints[i].x) < 0.5f &&
19                     std::abs(kp.pt.y - matched2DPoints[i].y) < 0.5f)
20                 {
21                     float w = getSemanticWeight(region.cls_id) *
22                         permanentWeight;
23                     weights[i] = std::max(weights[i], w); // keep the
24                     // highest
25                     break;
26                 }
27             }
28         }
29     }
30     return weights;
31 }

```

Listing 6: assignWeightsFromLandmarks() — relocalization.cpp, lines 1170–1194

Table 3: Weight assignment parameters (relocalization_node.cpp, lines 209–215)

Variable	Type	Value	Role
permanentWeight	float	1.0	Multiplier on getSemanticWeight() result
backgroundWeight	float	0.3	Default weight outside any bbox
Proximity tolerance	float	0.5 px	Max distance to match a 2D point to a keypoint

§7 Weighted PnP — Mathematical Formulation

7.1 Objective Function

Given n 2D–3D correspondences $\{(\mathbf{p}_i, \mathbf{X}_i^w)\}$ and per-point weights $\{w_i\}$, the goal is to find the camera pose $\mathbf{T}_{cw} \in SE(3)$ that minimises the *weighted reprojection error*:

Weighted Cost Function

$$E(\xi) = \sum_{i=1}^n w_i \|\mathbf{r}_i\|^2 = \sum_{i=1}^n w_i \|\mathbf{p}_i - \pi(\mathbf{T}_{cw} \cdot \mathbf{X}_i^w)\|^2 \quad (3)$$

$\xi \in \mathfrak{se}(3)$	6-D Lie algebra element (3 translation + 3 rotation)
$w_i \in \{0.30, 0.70, 1.00\}$	Semantic weight
$\mathbf{p}_i \in \mathbb{R}^2$	Observed 2-D keypoint
$\pi(\cdot)$	Perspective projection (Eq. 1)
$\mathbf{T}_{cw} \in SE(3)$	Camera-to-world transformation
$\mathbf{X}_i^w \in \mathbb{R}^3$	Map 3-D point
$\mathbf{r}_i \in \mathbb{R}^2$	Reprojection residual

7.2 SE(3) Lie Group Representation

The pose is stored as a Sophus `SE3d` object. Rather than operating in Euclidean space, updates are applied on the manifold via the *exponential map* (left perturbation):

Pose Update via Exponential Map

$$\mathbf{T}_{\text{new}} = \exp(\delta\xi) \cdot \mathbf{T}_{cw}, \quad \delta\xi \in \mathbb{R}^6 \quad (4)$$

$\delta\xi = [\delta\mathbf{t}^\top, \delta\boldsymbol{\omega}^\top]^\top$: 3 translational + 3 rotational increments.

7.3 Weight Normalisation

Before optimization, all weights are normalised so their mean equals 1. This keeps the scale of the Hessian $\mathbf{H}$ consistent across frames with different numbers of correspondences:

Weight Normalisation

$$\bar{w} = \frac{1}{n} \sum_{i=1}^n w_i, \quad \hat{w}_i = \frac{w_i}{\bar{w}} \quad (5)$$

```

1 float w_sum = 0.0f;
2 for (float w : weights) w_sum += w;
3 float w_mean = w_sum / n;
4
5 std::vector<double> w_norm(n);
6 for (int i = 0; i < n; ++i)
7     w_norm[i] = (double)(weights[i] / w_mean);

```

Listing 7: Weight normalisation — `relocalization.cpp`, lines 1218–1226

§8 Projection Jacobian

The Gauss–Newton solver needs the Jacobian of the 2-D residual $\mathbf{r}_i$ with respect to the 6-D perturbation $\delta\xi$:

$$\mathbf{J}_i = \frac{\partial \mathbf{r}_i}{\partial \delta\xi} = \frac{\partial [u, v]^\top}{\partial [\delta\mathbf{t}^\top, \delta\boldsymbol{\omega}^\top]} \in \mathbb{R}^{2 \times 6} \quad (6)$$

Let $\mathbf{P}_c = [X_c, Y_c, Z_c]^\top$ be the point in camera coordinates, $\zeta_1 = Z_c^{-1}$, $\zeta_2 = Z_c^{-2}$. The full Jacobian under the left SE(3) perturbation model is:

2 × 6 Projection Jacobian

$$\mathbf{J} = \begin{pmatrix} f_x \zeta_1 & 0 & -f_x X_c \zeta_2 & -f_x X_c Y_c \zeta_2 & f_x(1 + X_c^2 \zeta_2) & -f_x Y_c \zeta_1 \\ 0 & f_y \zeta_1 & -f_y Y_c \zeta_2 & -f_y(1 + Y_c^2 \zeta_2) & f_y X_c Y_c \zeta_2 & f_y X_c \zeta_1 \end{pmatrix} \quad (7)$$

Columns: $[\partial/\partial t_x, \partial/\partial t_y, \partial/\partial t_z, \partial/\partial \omega_x, \partial/\partial \omega_y, \partial/\partial \omega_z]$.

```

1 Eigen::Matrix<double, 2, 6> RelocalizationModule::
  computeProjectionJacobian(
2     const Eigen::Vector3d &Pc, double fx, double fy)
3 {
4     double Xc = Pc(0), Yc = Pc(1), Zc = Pc(2);
5     double Zc_inv = 1.0 / Zc;
6     double Zc_inv2 = Zc_inv * Zc_inv;
7
8     Eigen::Matrix<double, 2, 6> J;
9
10    // Row 0: du/d(delta_xi) = [du/dt | du/domega]
11    J(0, 0) = fx * Zc_inv;
12    J(0, 1) = 0.0;
13    J(0, 2) = -fx * Xc * Zc_inv2;
14    J(0, 3) = -fx * Xc * Yc * Zc_inv2;
15    J(0, 4) = fx + fx * Xc * Xc * Zc_inv2;
16    J(0, 5) = -fx * Yc * Zc_inv;
17
18    // Row 1: dv/d(delta_xi) = [dv/dt | dv/domega]
19    J(1, 0) = 0.0;
20    J(1, 1) = fy * Zc_inv;
21    J(1, 2) = -fy * Yc * Zc_inv2;
22    J(1, 3) = -(fy + fy * Yc * Yc * Zc_inv2);
23    J(1, 4) = fy * Xc * Yc * Zc_inv2;
24    J(1, 5) = fy * Xc * Zc_inv;
25
26    return J;
27 }

```

Listing 8: computeProjectionJacobian() — relocalization.cpp, lines 1124–1147

§9 Levenberg–Marquardt Solver

9.1 Weighted Normal Equations

At each iteration the weighted Gauss–Newton system is assembled and damped:

Weighted Normal Equations

$$\mathbf{H} = \sum_{i=1}^n \hat{w}_i \mathbf{J}_i^\top \mathbf{J}_i \in \mathbb{R}^{6 \times 6} \quad (8)$$

$$\mathbf{b} = \sum_{i=1}^n \hat{w}_i \mathbf{J}_i^\top \mathbf{r}_i \in \mathbb{R}^6 \quad (9)$$

$$\mathbf{H}_\lambda = \mathbf{H} + \lambda \text{diag}(\mathbf{H}), \quad \mathbf{H}_\lambda \delta \xi = \mathbf{b} \quad (10)$$

The damped system (Eq. 10) is solved with LDLT decomposition.

The LM damping parameter λ adapts every iteration:

if $E_{\text{new}} < E_{\text{old}} \Rightarrow \lambda /= 10$ (more Newton-like)

if $E_{\text{new}} \geq E_{\text{old}} \Rightarrow \lambda *= 10$ (more gradient-descent-like)

9.2 Core Optimization Loop

```

1 Sophus::SE3d T_cw(R_e, t_e); // warm-started from EPnP
2 double lambda = 1e-3;
3 double cost = std::numeric_limits<double>::max();
4
5 for (int iter = 0; iter < maxIterations; ++iter)
6 {
7     Eigen::Matrix<double,6,6> H = Eigen::Matrix<double,6,6>::Zero();
8     Eigen::Matrix<double,6,1> b = Eigen::Matrix<double,6,1>::Zero();
9     double new_cost = 0.0;
10
11     for (int i = 0; i < n; ++i)
12     {
13         Eigen::Vector3d Xw(points3D[i].x, points3D[i].y, points3D[i].z);
14         Eigen::Vector3d Pc = T_cw * Xw;
15         if (Pc(2) <= 0) continue; // skip points behind camera
16
17         double u_proj = fx * Pc(0) / Pc(2) + cx;
18         double v_proj = fy * Pc(1) / Pc(2) + cy;
19         Eigen::Vector2d r(points2D[i].x - u_proj, points2D[i].y - v_proj);
20
21         Eigen::Matrix<double,2,6> J = computeProjectionJacobian(Pc, fx,
22             fy);
23         double w = w_norm[i]; // normalised semantic weight
24         H += w * J.transpose() * J;
25         b += w * J.transpose() * r;
26         new_cost += w * r.squaredNorm();
27     }
28
29     // Levenberg-Marquardt damping
30     Eigen::Matrix<double,6,6> H_d = H;
31     for (int k = 0; k < 6; ++k)
32         H_d(k,k) += lambda * H(k,k);
33
34     Eigen::Matrix<double,6,1> delta = H_d.ldlt().solve(b);
35     Sophus::SE3d T_new = Sophus::SE3d::exp(delta) * T_cw;
36
37     if (new_cost < cost) { T_cw = T_new; cost = new_cost; lambda /=
38         10.0; }
39     else { lambda *= 10.0; }
40
41     if (delta.norm() < convergenceThreshold) break;
42 }

```

Listing 9: LM core loop — relocalization.cpp, lines 1254–1302 (abridged)

9.3 Solver Parameters

Table 4: `solvePnPWeighted()` parameters (`relocalization.h`, lines 87–93)

Parameter	Type	Default	Meaning
<code>maxIterations</code>	<code>int</code>	100	Maximum LM iterations
<code>convergenceThreshold</code>	<code>double</code>	1e-6	Stop when $\ \delta\xi\ < \text{this}$
<code>inlierThresholdPx</code>	<code>float</code>	2.0	Reprojection cutoff for inlier count (px)
λ_0	<code>double</code>	1e-3	Initial LM damping

EPnP Warm-Start

Before the LM loop, `solvePnPWeighted()` calls `cv::solvePnP` with flag `SOLVEPNP_EPNP` to obtain an initial $\mathbf{T}_{cw}$. This prevents cold-start divergence and typically halves the number of LM iterations needed.

10 Result Structure and Metrics

The solver returns a `WeightedPnPResult` struct:

```

1 struct WeightedPnPResult
2 {
3     bool          success;                // converged with >=
4     cv::Point3f    position;              // camera position in world
5     int           numInliers;             // points with reproj. error <
6     int           totalCorrespondences;   // total matched 2D-3D pairs
7     float         meanReprojectionError;  // unweighted mean reproj.
8     float         weightedReprojectionError; // weighted cost / n
9     int           iterations;             // LM iterations to convergence
10    std::vector<int> inlierIndices;        // accepted inlier indices
11    cv::Mat        rvec;                  // Rodrigues rotation vector
12    cv::Mat        tvec;                  // translation vector
13 };

```

Listing 10: `WeightedPnPResult` — `relocalization.h`, lines 29–41

The node also writes a per-frame CSV (`comparison_log.csv`) to enable offline comparison of weighted vs. standard RANSAC-PnP:

Table 5: CSV log columns for weighted vs. standard PnP comparison

Column	Meaning
<code>std_inliers</code>	Standard RANSAC-PnP inlier count
<code>wnp_inliers</code>	Weighted-PnP inlier count
<code>wnp_reproj_px</code>	Weighted-PnP mean reprojection error (px)
<code>wnp_weighted_cost</code>	Final cost $E(\xi)$ (Eq. 3)
<code>pose_delta_m</code>	Translation difference vs. standard PnP (m)
<code>wnp_iterations</code>	LM iterations used

§11 Full Algorithm Flowchart

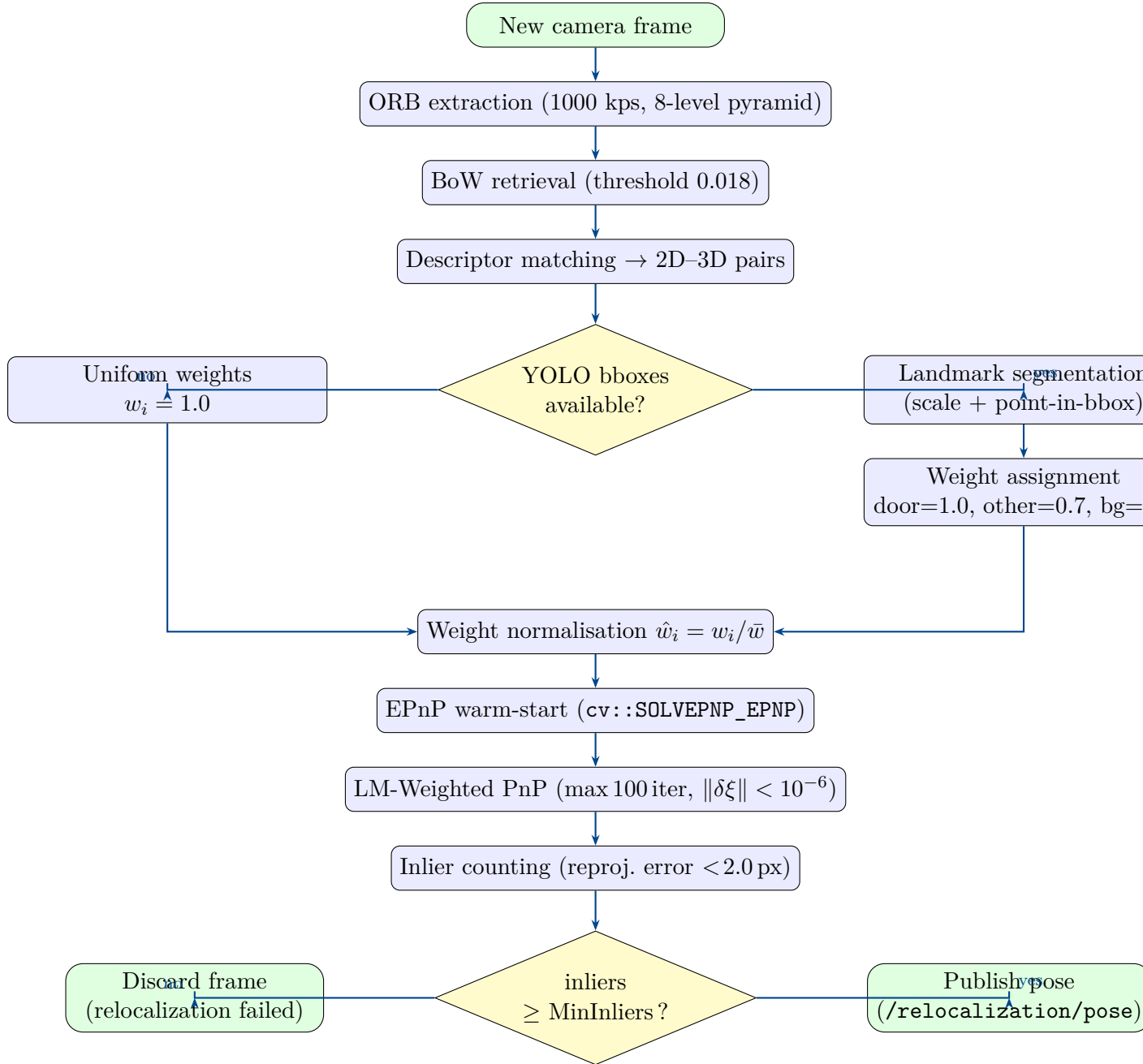


Figure 5: End-to-end algorithm flow for one camera frame. When no YOLO bboxes are available, all matches receive unit weight and the solver reduces to standard (unweighted) Gauss–Newton PnP.

Summary of Key Takeaways

Key Takeaways

1. **Semantic weighting** raises the influence of permanent structural landmarks (doors) and suppresses unreliable background features, guided purely by YOLO bounding boxes — no 3-D landmark model is needed.
2. **SE(3) Lie-group** optimization via Sophus avoids singularities from Euler-angle or quaternion parameterizations and guarantees that every iterate is a valid rigid body transformation.
3. **Levenberg–Marquardt** adapts smoothly between gradient descent (far from minimum, large λ) and Newton steps (near minimum, small λ), giving both global robustness and fast local convergence.
4. **EPnP warm-start** initialises the LM solver close to the correct pose, preventing divergence in scenes with many outliers.
5. **Normalised weights** $\hat{w}_i = w_i/\bar{w}$ keep the Hessian $\mathbf{H}$ well-conditioned regardless of how many correspondences the current frame produces.